

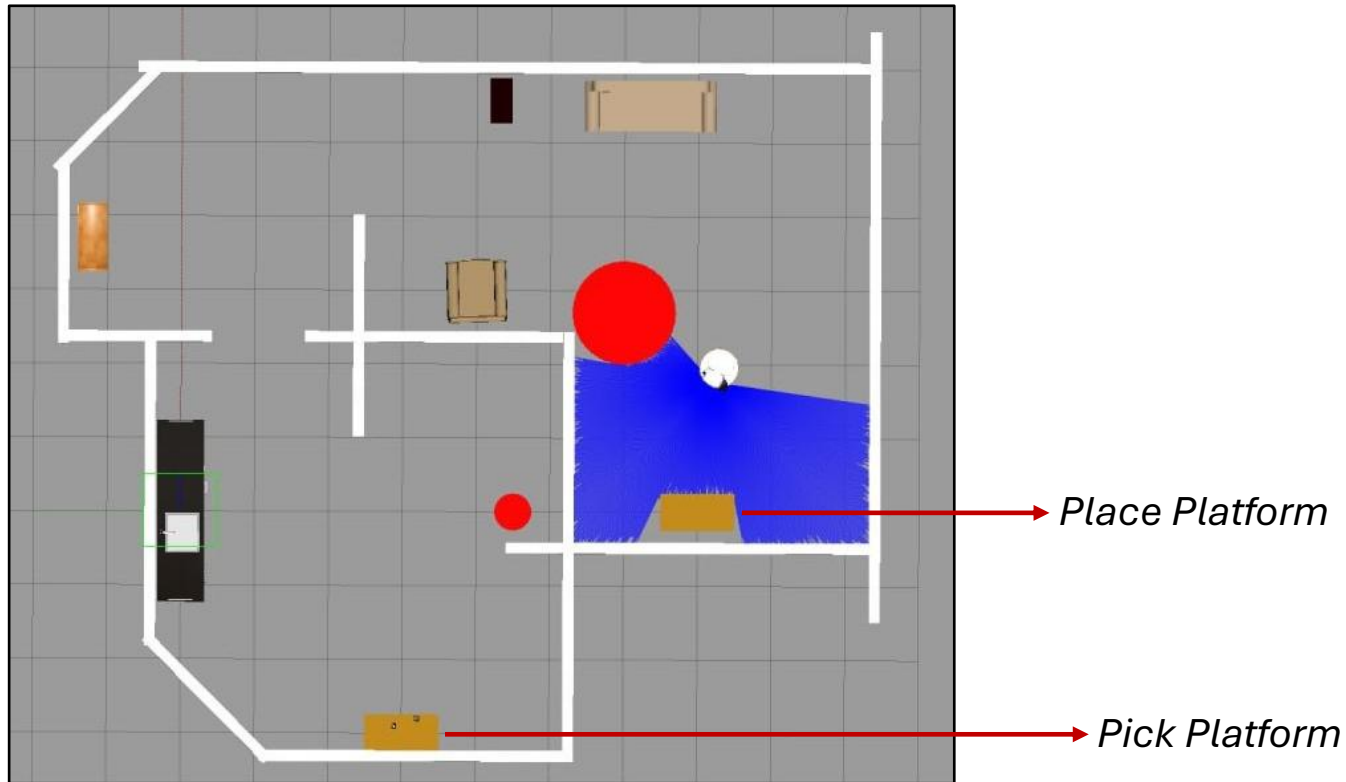
AMR Tiago Project Report

Vincenzo Lombardi – Group 7
Academic Year 2025/2026



Project Setup & Objectives/Tasks

- The **projects setup** is the one provided during the lessons (in terms of files), and with the following home/world (**group7**):



- The **projects objectives** are clearly explained in the assignment files as divided in **three Tasks**.

Project Setup & Objectives/Tasks

- The **projects setup** is the one provided during the lessons (in terms of files), and with the following home/world (**group7**):



some slight modifications, for example:

*Tiago spawn location can now
be provided as a parameter
(launch argument)
(not anymore hard-coded)*

*The default topic into which
velocity is written (**cmd_vel**)
can now be renamed at will
(used in Task2,
for Collision Monitor)*

- The **projects objectives** are clearly explained in the assignment files as divided in **three Tasks**.

Project Files Structure

- **tiago_ws/src/tiago_exam_interfaces**: defines the [interfaces](#) for the [TiagoArm](#) and [TiagoGripper](#) Action Servers.
- **tiago_ws/src/tiago_exam_tasks/launch**: contains the [three launch files](#) to be used for the three Project Tasks.
- **tiago_ws/src/tiago_exam_tasks/tiago_exam_tasks/nodes**: contains...
 - the [nodes](#) related to the three [FSM](#) for the three Tasks
 - the [node](#) exposing the [TiagoArm](#) Action Server, *to be used to control the Tiago Arm configuration or EE-pose*
 - the [node](#) exposing the [TiagoGripper](#) Action Server, *to be used to control the Tiago Gripper in opening or closing*
 - the [discoveryPlanner.py](#) file to be used in the Task2 to plan the discovery procedure
 - the [discoveryLoader.py](#) file to be used in the Task3 to load the results of the discovery (AKA pick/place locations)
 - the [tiagoExamParameters.py](#) file, containing parameters values used in the whole Project
- **tiago_ws/src/m-explore-ros2** contains the [ExploreLite](#) package, **tiago_ws/src/IFRA_LinkAttacher** the [LinkAttacher](#) one
- **tiago_ws/src/pal_navigation_cfg_public/pal_navigation_cfg_params/params/tiago_nav2.yaml**: to tune Nav2 parameters
- *Of course, they are also exploited all the files provided as the Project setup, contained in **ros2_ws** and **tiago_ws**, for example:*
[tiago_ws/src/tiago_exam/launch/tiago_exam.launch.py](#)
[tiago_ws/src/tiago_navigation/tiago_2dnav/launch/tiago_nav_bringup.launch.py](#)
A note: *for an insight about these two files, go read the content of the launch files of the various Tasks (in particular, 1 and 2)*

TiagoArm & TiagoGripper Action Servers

- In the Project, they are present two simple nodes that expose two useful Action Servers:

➔ `tiago_ws\src\tiago_exam_tasks\tiago_exam_tasks\nodes\tiagoarm_node.py` 

The TiagoArm Action Server exploits the pymoveit2 library (as seen during lessons) to control the **arm** of the Tiago.

Two kind of usages are possible:

- provide a **pose** for the arm EE (*end effector*) frame in terms of position (the coordinates) and orientation (a quaternion)
- provide a **configuration** for the whole arm joints (*also including the torso*)

➔ `tiago_ws\src\tiago_exam_tasks\tiago_exam_tasks\nodes\tiagogripper_node.py` 

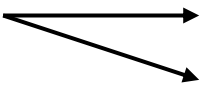
The TiagoGripper Action Server exploits the pymoveit2 library (as seen during lessons) to control the **gripper** of the Tiago.

The usage is simple: command open (full) or close (full).

Additionally, they can also be provided parameters for **LinkAttacher** and **LinkDetacher**. In that case:

- in case of an opening command, we **firstly detach** the (eventually) attached object.
- in case of a closing command, we do *not* perform a full closure: instead, we bring the gripper to the **grasping closure**.
After that, we **then attach** the indicated object (*all of that is tuned to grasp the cubes, that are of known dimensions*).

Task1 Launch File

- Task1 is about **autonomous exploration and mapping**
- The Launch file is responsible for launching all the components (other launch files, nodes, etc.) required for the task to function fully and correctly (and to set the related parameters), specifically including:
 - all proper **parameters** are set (*Tiago spawn location, Gazebo world name, FSM1 required parameters...*)
 - the two launch files to bringup the whole simulation are included (*with the proper configuration, indicated in the assignment*)
tiago_ws\src\tiago_exam\launch\tiago_exam.launch.py
tiago_ws\src\tiago_navigation\tiago_2dnav\launch\tiago_nav_bringup.launch.py
 - launch file for **ExploreLite** 
some modifications
 - not anymore starting automatically*
 - before sending a goal, computing the GlobalPath and spinning Tiago towards it (as in Task2/3)*
 - the **node** that implements the **FSM1**, governing the behaviour of the Task1
 - the **node** that exposes the **TiagoArm Action Server**, exploited to tackle the arm before starting the exploration

```
▼ tiago_exam_tasks
  > config
  ▼ launch
    • task1.launch.py
    • task2.launch.py
    • task3.launch.py
```

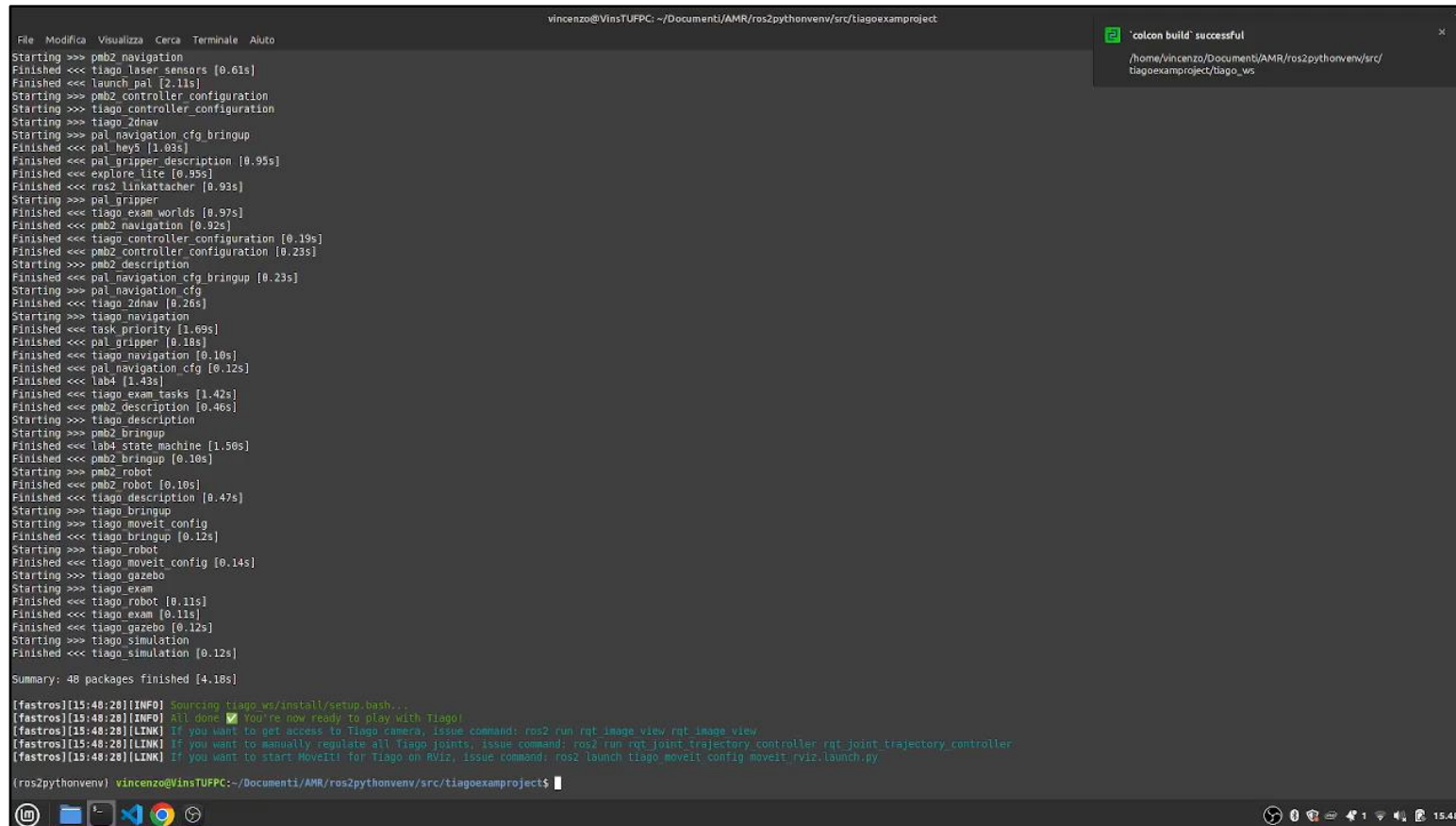
Task1 Behaviour (FSM)

- A simple node implementing the Task1 behaviour as a FSM, exploiting proper topics/services/actions (the usage of these elements has been implemented accordingly to the official ROS2 Humble documentation).

Arm tucking (TiagoArm Action Server)

Exploring (exploiting ExploreLite)

Saving obtained map (SaveMap Service)



```
File Modifica Visualizza Cerca Terminale Aiuto
vincenzo@VinsTUFP: ~/Documents/AMR/ros2pythonenv/src/tiagoexamproject

Starting >>> pmb2 navigation
Finished <<< tiago_laser_sensors [0.61s]
Finished <<< launch_pal [2.11s]
Starting >>> pmb2 controller configuration
Starting >>> tiago controller configuration
Starting >>> tiago_2dnav
Starting >>> pal_navigation_cfg_bringup
Finished <<< pal_hey5 [1.03s]
Finished <<< pal_gripper_description [0.95s]
Finished <<< explore_lite [0.95s]
Finished <<< ros2_linkattacher [0.93s]
Starting >>> pal_gripper
Finished <<< tiago_exam_worlds [0.97s]
Finished <<< pmb2 navigation [0.92s]
Finished <<< tiago_controller_configuration [0.19s]
Finished <<< pmb2 controller configuration [0.23s]
Starting >>> pmb2 description
Finished <<< pal_navigation_cfg_bringup [0.23s]
Starting >>> pal_navigation_cfg
Finished <<< tiago_2dnav [0.26s]
Starting >>> tiago_navigation
Finished <<< task_priority [1.09s]
Finished <<< pal_gripper [0.18s]
Finished <<< tiago_navigation [0.10s]
Finished <<< pal_navigation_cfg [0.12s]
Finished <<< lab4 [1.42s]
Finished <<< tiago_exam_tasks [1.42s]
Finished <<< pmb2 description [0.46s]
Starting >>> tiago_description
Starting >>> pmb2 bringup
Finished <<< lab4 state machine [1.50s]
Finished <<< pmb2 bringup [0.10s]
Starting >>> pmb2 robot
Finished <<< pmb2 robot [0.10s]
Finished <<< tiago_description [0.47s]
Starting >>> tiago bringup
Starting >>> tiago moveit config
Finished <<< tiago bringup [0.12s]
Starting >>> tiago robot
Finished <<< tiago moveit config [0.14s]
Starting >>> tiago gazebo
Starting >>> tiago_exam
Finished <<< tiago robot [0.11s]
Finished <<< tiago_exam [0.11s]
Finished <<< tiago_gazebo [0.12s]
Starting >>> tiago simulation
Finished <<< tiago_simulation [0.12s]

Summary: 40 packages finished [4.18s]

[fastros][15:40:28][INFO] Sourcing tiago_ws/install/setup.bash...
[fastros][15:40:28][INFO] All done! You're now ready to play with Tiago!
[fastros][15:40:28][LINK] If you want to get access to Tiago camera, issue command: ros2 run rqt_image_view rqt_image_view
[fastros][15:40:28][LINK] If you want to manually regulate all Tiago joints, issue command: ros2 run rqt_joint_trajectory_controller rqt_joint_trajectory_controller
[fastros][15:40:28][LINK] If you want to start MoveIt! for Tiago on RViz, issue command: ros2 launch tiago_moveit_config moveit_rviz.launch.py

(ros2pythonenv) vincenzo@VinsTUFP:~/Documents/AMR/ros2pythonenv/src/tiagoexamproject$
```

colcon build successful
/home/vincenzo/Documents/AMR/ros2pythonenv/src/tiagoexamproject/tiago_ws

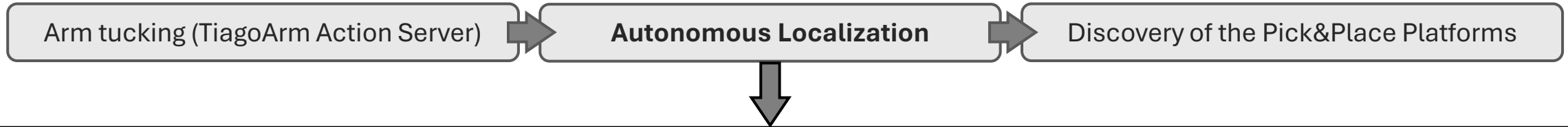
Task2 Launch File

- Task2 is about **autonomous localization and platforms (pick & place) discovery** (exploiting the map from Task1)
- The Launch file is responsible for launching all the components (other launch files, nodes, etc.) required for the task to function fully and correctly (and to set the related parameters), specifically including:
 - all proper **parameters** are set (*different Tiago spawn location, Gazebo world name, FSM2 required parameters...*)
 - the two launch files to bringup the whole simulation are again included (*again, configured as indicated in the assignment*)
 - the linear Tiago velocity is not anymore written in **cmd_vel** (final actuation topic): the information flow is redirected to the topic **cmd_vel_unsafe** (the **Collision Monitor** will then perform **cmd_vel_unsafe** → **cmd_vel_safe** → **cmd_vel**) 🔍
 - the **node** that exposes the **TiagoArm Action Server**, exploited to tackle the arm before starting localization & discovery
 - the **node** that implements the Ros2 Humble **Collision Monitor** (*LifecycleNode* ⇒ need for a *LifeCycleManager* to active it)
 - Collision Monitor configuration can be found at **tiago_ws\src\tiago_exam_tasks\config\collision_monitor.yaml**
 - the **node** related to **Nav2LifecycleManager**, required for the Collision Monitor activation (*accordingly to the Ros2 Humble docs*)
 - Collision Monitor is brought toward: *unconfigured* → *configured but inactive* → *configured and active* (but internally disabled)
 - the **node** that implements the **Aruco Marker detection** for ID 26 (*pick location*)
 - the **node** that implements the **Aruco Marker detection** for ID 238 (*place location*)
 - the **node** that implements the **FSM2**, governing the behaviour of the Task2

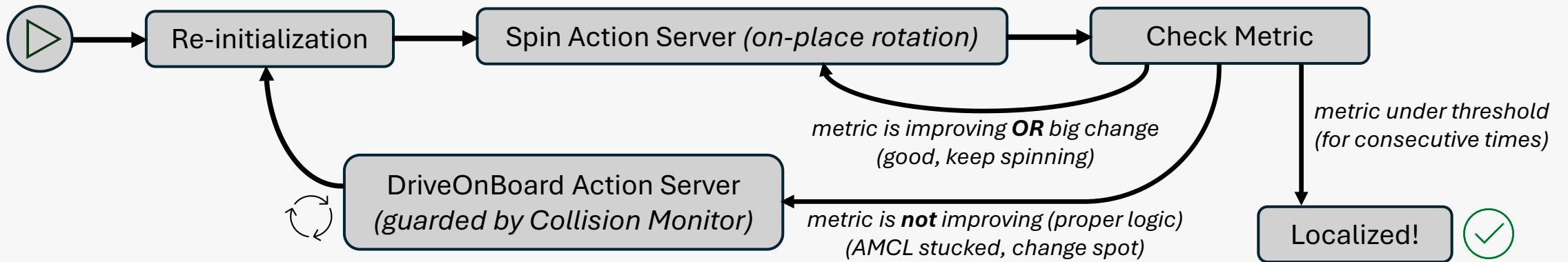
```
▼ tiago_exam_tasks
  > config
  ▼ launch
    📄 task1.launch.py
    📄 task2.launch.py
    📄 task3.launch.py
```


Task2 Behaviour (FSM)

- A simple node implementing the Task2 behaviour as a FSM, exploiting proper topics/services/actions (the usage of these elements has been implemented accordingly to the official ROS2 Humble documentation).

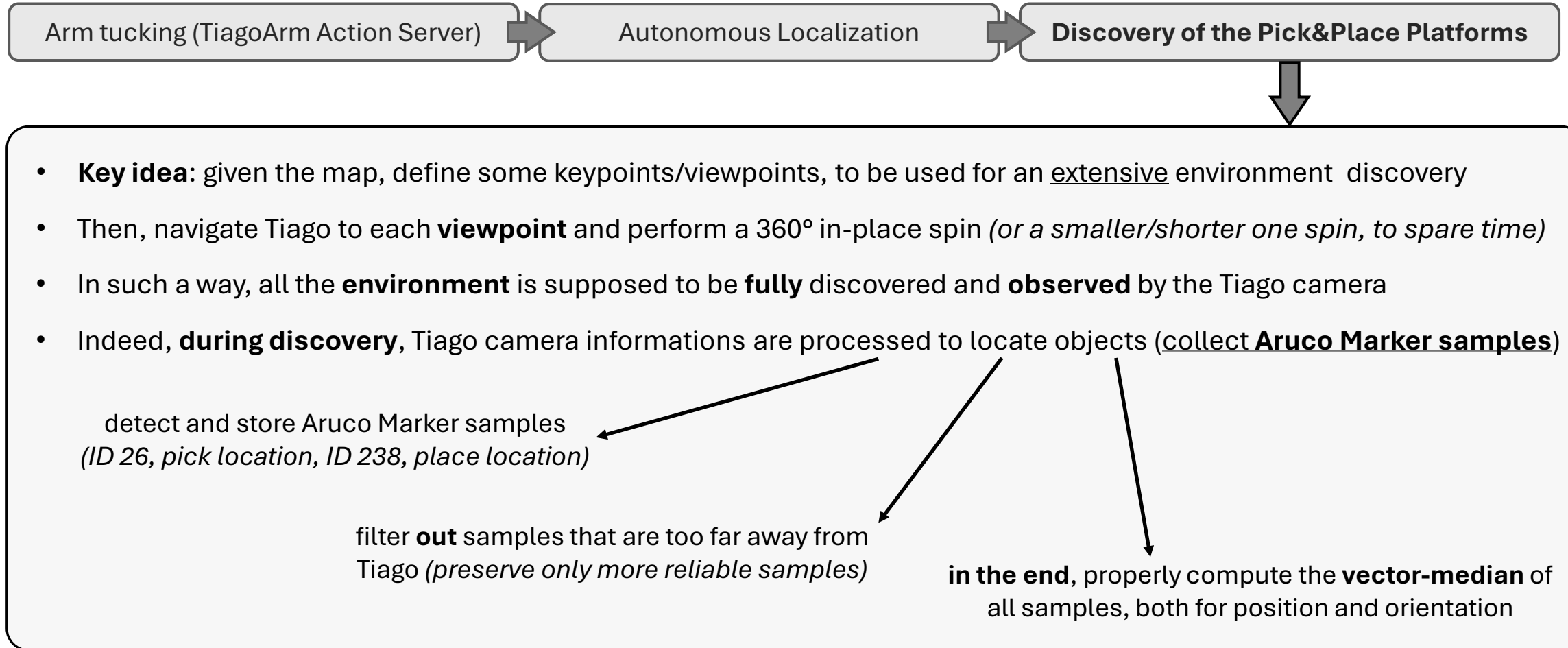


- Exploiting **AMCL**, as seen during the course lessons
- Using the `/reinitialize_global_localization` Service: *re-inizialization* of the particles distribution & probability accross the map
- Localization metric $M = \sigma_x^2 + \sigma_y^2 + \sigma_{yaw}^2$ (exploiting the AMCL covariance 6x6 matrix in its first, second and sixth diagonal values)



Task2 Behaviour (FSM)

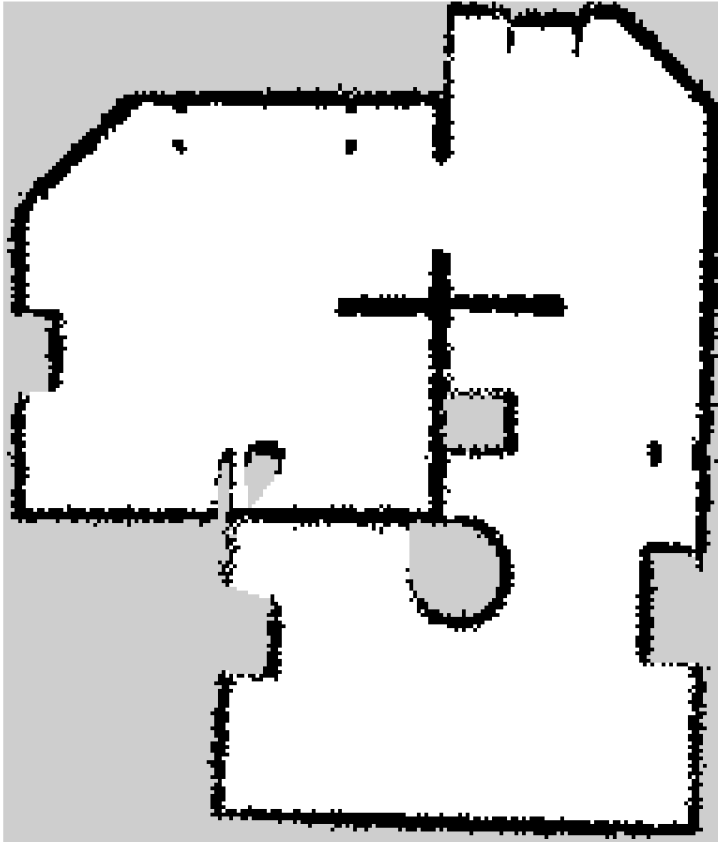
- A simple node implementing the Task2 behaviour as a FSM, exploiting proper topics/services/actions (the usage of these elements has been implemented accordingly to the official ROS2 Humble documentation).



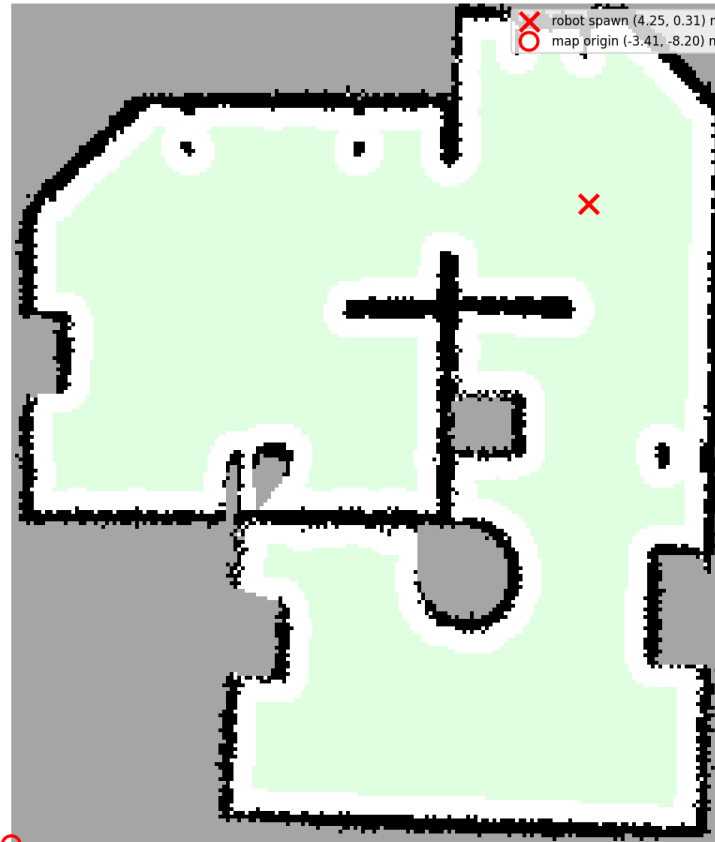
Task2 Behaviour (FSM)

Viewpoints are computed from the skeleton of the map green-space, in terms of junctions **and/or** backtracked keypoints.
In the final version: at each viewpoint, not a full rotation, but the longest rotation towards the GlobalPath toward the new viewpoint.

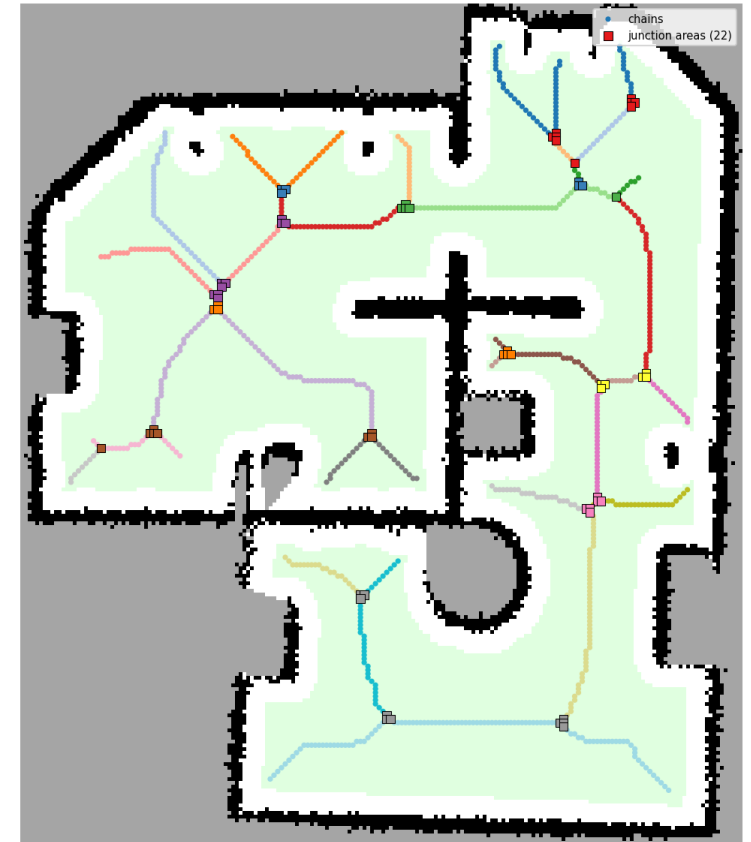
1: original map (raw PGM)



2: green space + robot spawn + map origin



3: skeleton chains + junction areas

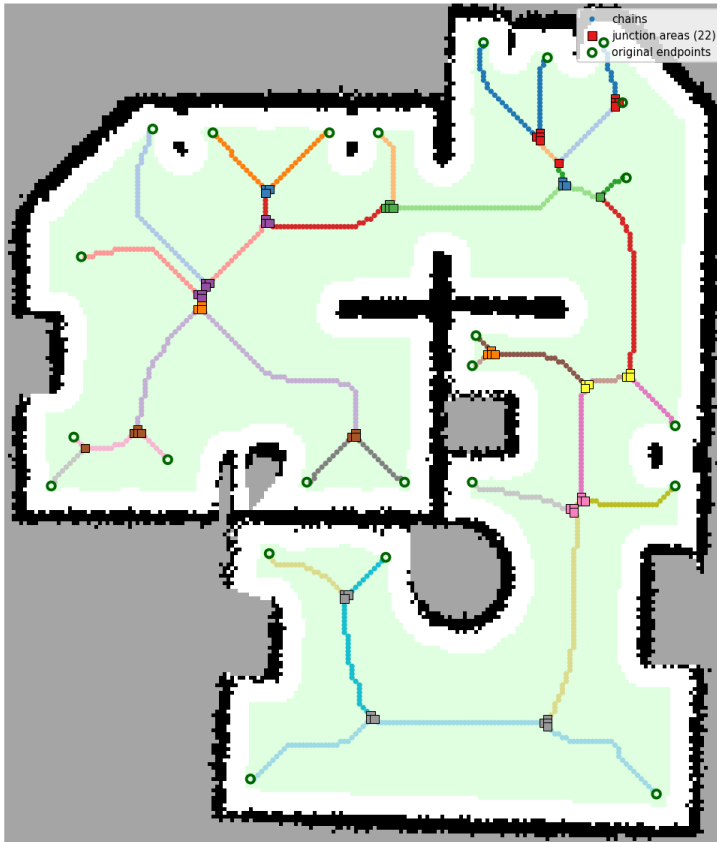


discoveryPlanner.py file encapsulates the code for all viewpoints computation

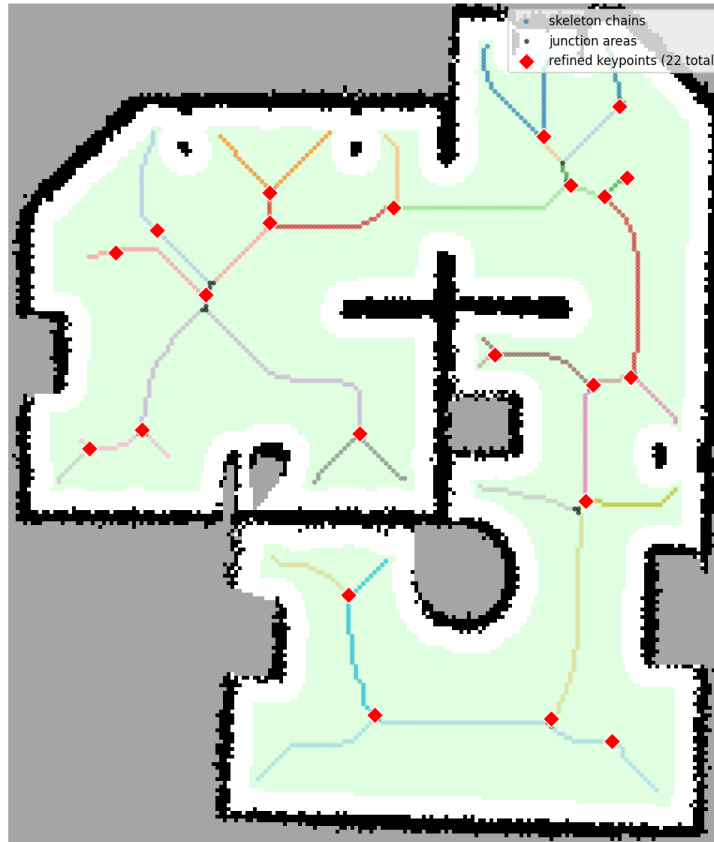
Task2 Behaviour (FSM)

Viewpoints are computed from the skeleton of the map green-space, in terms of junctions **and/or** backtracked keypoints. In the final version: at each viewpoint, not a full rotation, but the longest rotation towards the GlobalPath toward the new viewpoint.

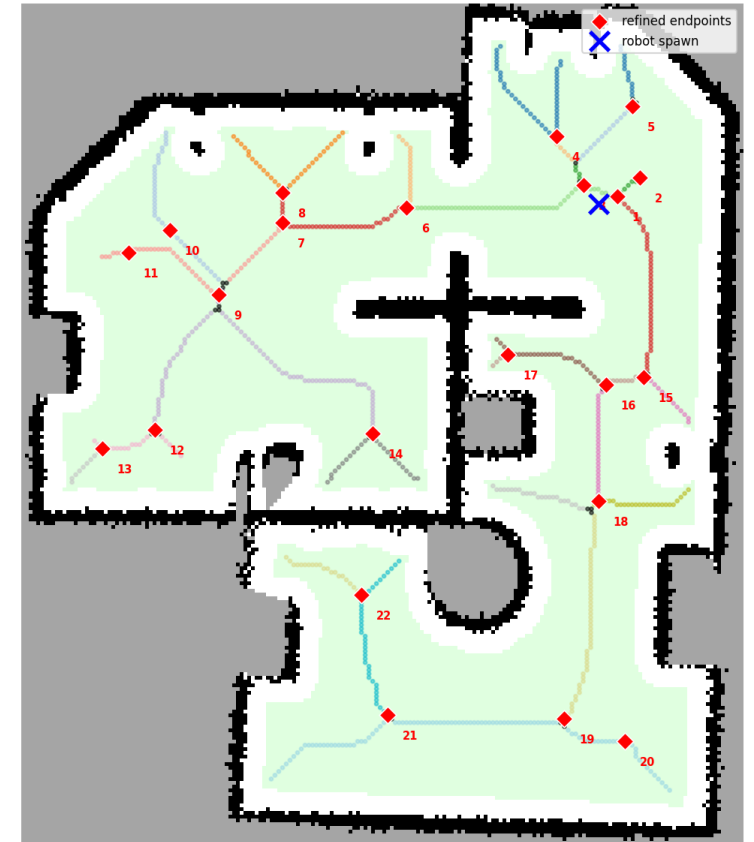
4: original keypoints (clearance threshold 50% → 1.054 m)



5: refined keypoints (clearance threshold 50% → 1.054 m)



6: ordered refined keypoints + robot spawn



discoveryPlanner.py file encapsulates the code for all viewpoints computation

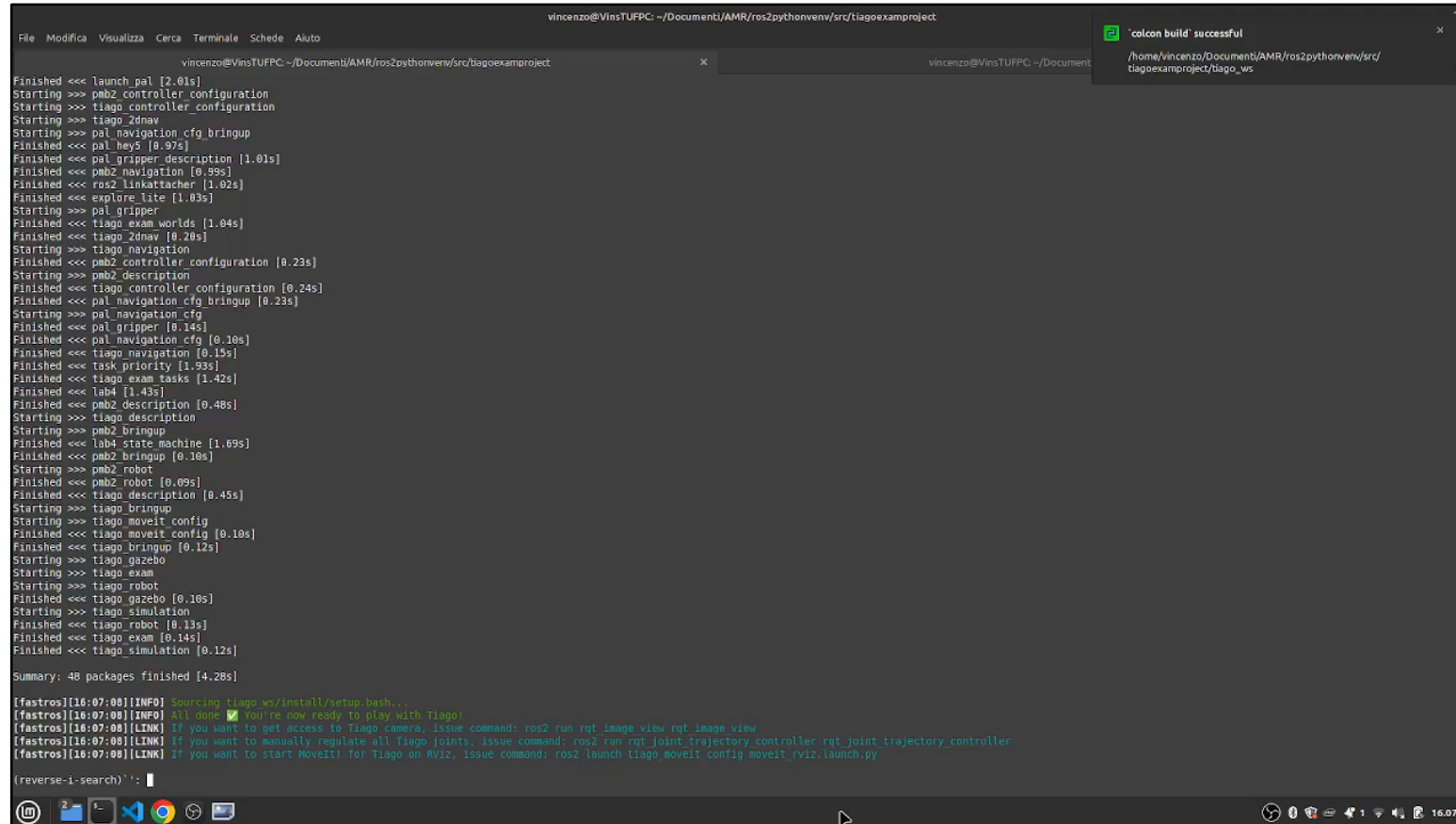
Task2 Behaviour (FSM)

- A simple node implementing the Task2 behaviour as a FSM, exploiting proper topics/services/actions (the usage of these elements has been implemented accordingly to the official ROS2 Humble documentation).

Arm tucking (TiagoArm Action Server)

Autonomous Localization

Discovery of the Pick&Place Platforms



```
File Modifica Visualizza Cerca Terminale Schede Aiuto
vincenzo@VinsTUFP: ~/Documents/AMR/ros2pythonenv/src/tiagoexamproject
vincenzo@VinsTUFP: ~/Documents/AMR/ros2pythonenv/src/tiagoexamproject
Finished <<< launch_pal [2.01s]
Starting >>> pmb2_controller_configuration
Starting >>> tiago_controller_configuration
Starting >>> tiago_2dnav
Starting >>> pal_navigation_cfg_bringup
Finished <<< pal_heyls [0.97s]
Finished <<< pal_gripper_description [1.01s]
Finished <<< pmb2_navigation [0.99s]
Finished <<< ros2_linkattacher [1.02s]
Finished <<< explore_lite [1.03s]
Starting >>> pal_gripper
Finished <<< tiago_exam_worlds [1.04s]
Finished <<< tiago_2dnav [0.20s]
Starting >>> tiago_navigation
Finished <<< pmb2_controller_configuration [0.23s]
Starting >>> pmb2_description
Finished <<< tiago_controller_configuration [0.24s]
Finished <<< pal_navigation_cfg_bringup [0.23s]
Starting >>> pal_navigation_cfg
Finished <<< pal_gripper [0.14s]
Finished <<< pal_navigation_cfg [0.10s]
Finished <<< tiago_navigation [0.15s]
Finished <<< task_priority [1.93s]
Finished <<< tiago_exam_tasks [1.42s]
Finished <<< lab4 [1.43s]
Finished <<< pmb2_description [0.48s]
Starting >>> tiago_description
Starting >>> pmb2_bringup
Finished <<< lab4_state_machine [1.69s]
Finished <<< pmb2_bringup [0.10s]
Starting >>> pmb2_robot
Finished <<< pmb2_robot [0.09s]
Finished <<< tiago_description [0.45s]
Starting >>> tiago_bringup
Starting >>> tiago_moveit_config
Finished <<< tiago_moveit_config [0.10s]
Finished <<< tiago_bringup [0.12s]
Starting >>> tiago_gazebo
Starting >>> tiago_exam
Starting >>> tiago_robot
Finished <<< tiago_gazebo [0.10s]
Starting >>> tiago_simulation
Finished <<< tiago_robot [0.13s]
Finished <<< tiago_exam [0.14s]
Finished <<< tiago_simulation [0.12s]

Summary: 48 packages finished [4.28s]

[fastros][16:07:08][INFO] Sourcing tiago_ws/install/setup.bash...
[fastros][16:07:08][INFO] All done! You're now ready to play with Tiago!
[fastros][16:07:08][LINK] If you want to get access to Tiago camera, issue command: ros2 run rqt_image_view rqt_image_view
[fastros][16:07:08][LINK] If you want to manually regulate all Tiago joints, issue command: ros2 run rqt_joint_trajectory_controller rqt_joint_trajectory_controller
[fastros][16:07:08][LINK] If you want to start MoveIt! for Tiago on RViz, issue command: ros2 launch tiago_moveit_config moveit_rviz.launch.py

(reverse-i-search) '':
```

'colcon build' successful
/home/vincenzo/Documents/AMR/ros2pythonenv/src/tiagoexamproject/tiago_ws

Task2 Behaviour (FSM)

- Additional example:** autonomous localization initially stuck (*DriveOnBoard* and *AMCL* re-initialization are indeed performed) (*this example relies on a different map – new exploration & generation – w.r.t. the one shown before in the Task1 video*)

Arm tucking (TiagoArm Action Server)

Autonomous Localization

Discovery of the Pick&Place Platforms

```
File Modifica Visualizza Cerca Terminale Schede Aiuto
vincenzo@VinsTUFPCC: ~/Documenti/AMR/ros2pythonvenv/src/tiagoexamproject
vincenzo@VinsTUFPCC: ~/Documenti/AMR/ros2pythonvenv/src/tiagoexamproject
Finished <<< launch_pal [1.91s]
Starting >>> pmb2_controller_configuration
Starting >>> tiago_controller_configuration
Starting >>> tiago_2dnav
Starting >>> pal_navigation_cfg_bringup
Finished <<< pal_gripper_description [0.95s]
Finished <<< explore_lite [0.96s]
Finished <<< pmb2_navigation [1.09s]
Starting >>> pal_gripper
Finished <<< ros2_linkattacher [0.95s]
Finished <<< tiago_exam_worlds [0.95s]
Finished <<< pal_hey5 [0.94s]
Finished <<< tiago_controller_configuration [0.24s]
Finished <<< pmb2_controller_configuration [0.27s]
Starting >>> pmb2_description
Finished <<< tiago_2dnav [0.29s]
Starting >>> tiago_navigation
Finished <<< pal_navigation_cfg_bringup [0.29s]
Starting >>> pal_navigation_cfg
Finished <<< task_priority [1.77s]
Finished <<< pal_gripper [0.21s]
Finished <<< pal_navigation_cfg [0.10s]
Finished <<< tiago_navigation [0.13s]
Finished <<< lab4 [1.42s]
Finished <<< tiago_exam_tasks [1.47s]
Finished <<< pmb2_description [0.49s]
Starting >>> tiago_description
Starting >>> pmb2_bringup
Finished <<< lab4_state_machine [1.56s]
Finished <<< pmb2_bringup [0.10s]
Starting >>> pmb2_robot
Finished <<< pmb2_robot [0.11s]
Finished <<< tiago_description [0.48s]
Starting >>> tiago_bringup
Starting >>> tiago_moveit_config
Finished <<< tiago_bringup [0.11s]
Starting >>> tiago_robot
Finished <<< tiago_moveit_config [0.13s]
Starting >>> tiago_gazebo
Starting >>> tiago_exam
Finished <<< tiago_robot [0.12s]
Finished <<< tiago_gazebo [0.12s]
Starting >>> tiago_simulation
Finished <<< tiago_exam [0.13s]
Finished <<< tiago_simulation [0.14s]
Summary: 48 packages finished [4.29s]
[fastros][16:49:59][INFO] Sourcing tiago_ws/install/setup.bash...
[fastros][16:49:59][INFO] All done! You're now ready to play with Tiago!
[fastros][16:49:59][LINK] If you want to get access to Tiago camera, issue command: ros2 run rqt_image_view rqt_image_view
[fastros][16:49:59][LINK] If you want to manually regulate all Tiago joints, issue command: ros2 run rqt_joint_trajectory_controller rqt_joint_trajectory_controller
[fastros][16:49:59][LINK] If you want to start MoveIt! for Tiago on RViz, issue command: ros2 launch tiago_moveit_config moveit_rviz.launch.py
(vincenzo@VinsTUFPCC:~/Documenti/AMR/ros2pythonvenv/src/tiagoexamproject$ ros2 launch tiago_exam_tasks task1 launch.py
```

Task3 Launch File

- Task3 is about **sequential cubes transportation (63 first, then 58) from pick to place locations.**

Autonomous localization is again performed: in the Task3, the Tiago is still free to be spawned at a generic location, as well as it was both in Task1 and Task2 ⇒ Task3 is independent

Of course, the map (from Task1), also enriched with the notion of the pick & place locations (from Task2), is known and exploited. 

- The Launch file is responsible for launching all the components (other launch files, nodes, etc.) required for the task to function fully and correctly (and to set the related parameters), specifically including:

- all proper **parameters** are set (*Tiago spawn location, Gazebo world name, FSM3 required parameters...*)

- the two launch files to bringup the whole simulation are again included (*again, configured as indicated in the assignment*)

 *again Collision Monitor is exploited (for the autonomous localization)*

- again **nodes** for **TiagoArm Action Server**, **Collision Monitor** and **Nav2LifecycleManager** (indeed, as before)

- the **node** that exposes the **TiagoGripper Action Server**, exploited to control the Tiago gripper to grasp/release the cubes 

 *the cubes side is of dimension 7cm and the grasp closure is of 6.4cm (**LinkAttacher** is used)*

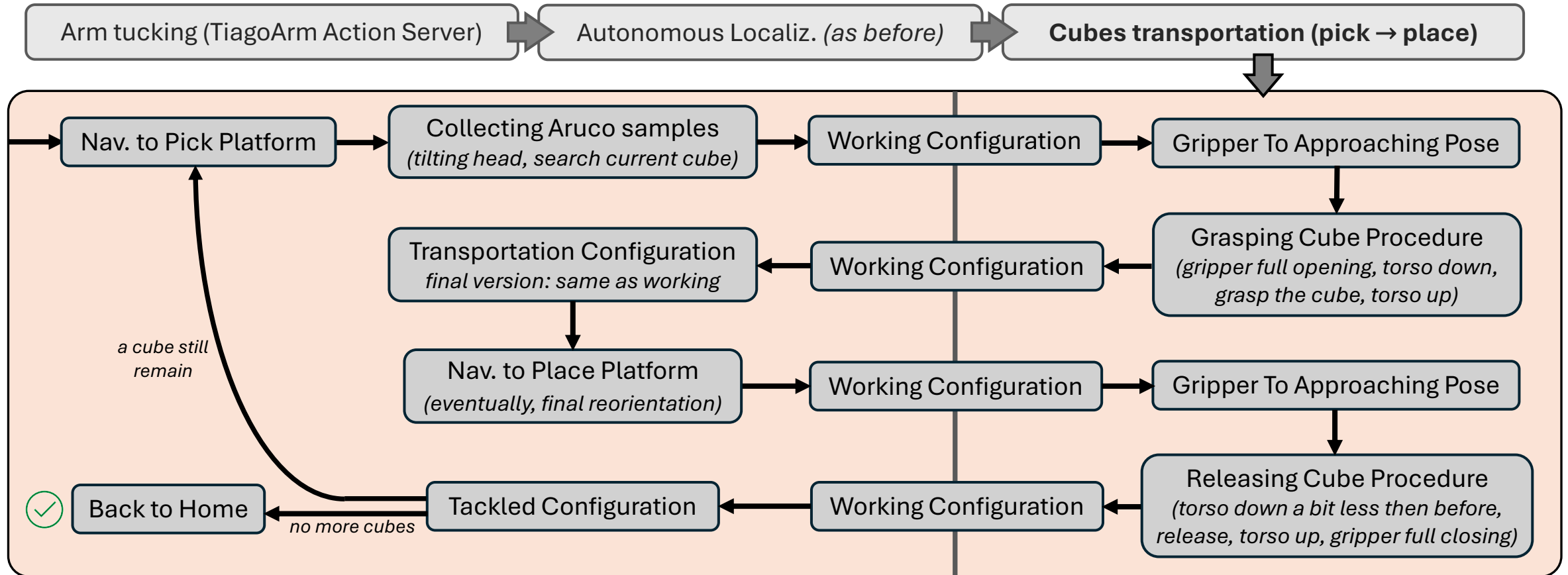
- The two **nodes** that implements the **Aruco Marker detection** for ID 63 (*first cube*) and ID 58 (*second cube*)

- the **node** that implements the **FSM3**, governing the behaviour of the Task3

```
▼ tiago_exam_tasks
  > config
  ▼ launch
    • task1.launch.py
    • task2.launch.py
    • task3.launch.py
```

Task2 Behaviour (FSM)

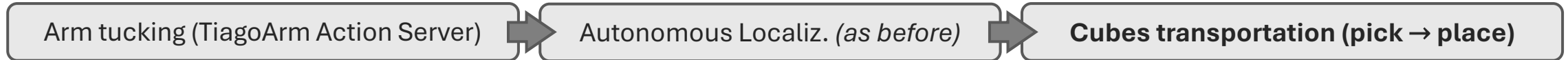
- A simple node implementing the Task3 behaviour as a FSM, exploiting proper topics/services/actions (the usage of these elements has been implemented accordingly to the [official ROS2 Humble documentation](#)).



The **Working Configuration** prepares the Tiago to approach the platform: the gripper is in a good position to be brought to the approaching pose, the torso is lifted up to be easily moved down later on, and the configuration guarantees avoiding hitting the platform itself

Task2 Behaviour (FSM)

- A simple node implementing the Task3 behaviour as a FSM, exploiting proper topics/services/actions (the usage of these elements has been implemented accordingly to the official ROS2 Humble documentation).



- Some **final remarks**:

- *as also done in the previous Tasks, before launching a new NavGoal, an in-place spin towards the GlobalPath is performed*
- *When **picking the cube**, the gripper **approaching pose** is computed exploiting:*
 - *a vector-median of the collected **cube marker samples only** in terms of its **position***
 - *the **orientation** is computed exploiting the one of the Tiago **base_link***
and it's of course computed in order to have the gripper just above the cube to be picked/grasped (after that, torso down)
- *When **placing the cube**, the same gripper **approaching pose** computed for the very same cube is exploited again (more specifically, it's moved toward the wall of a certain amount)*

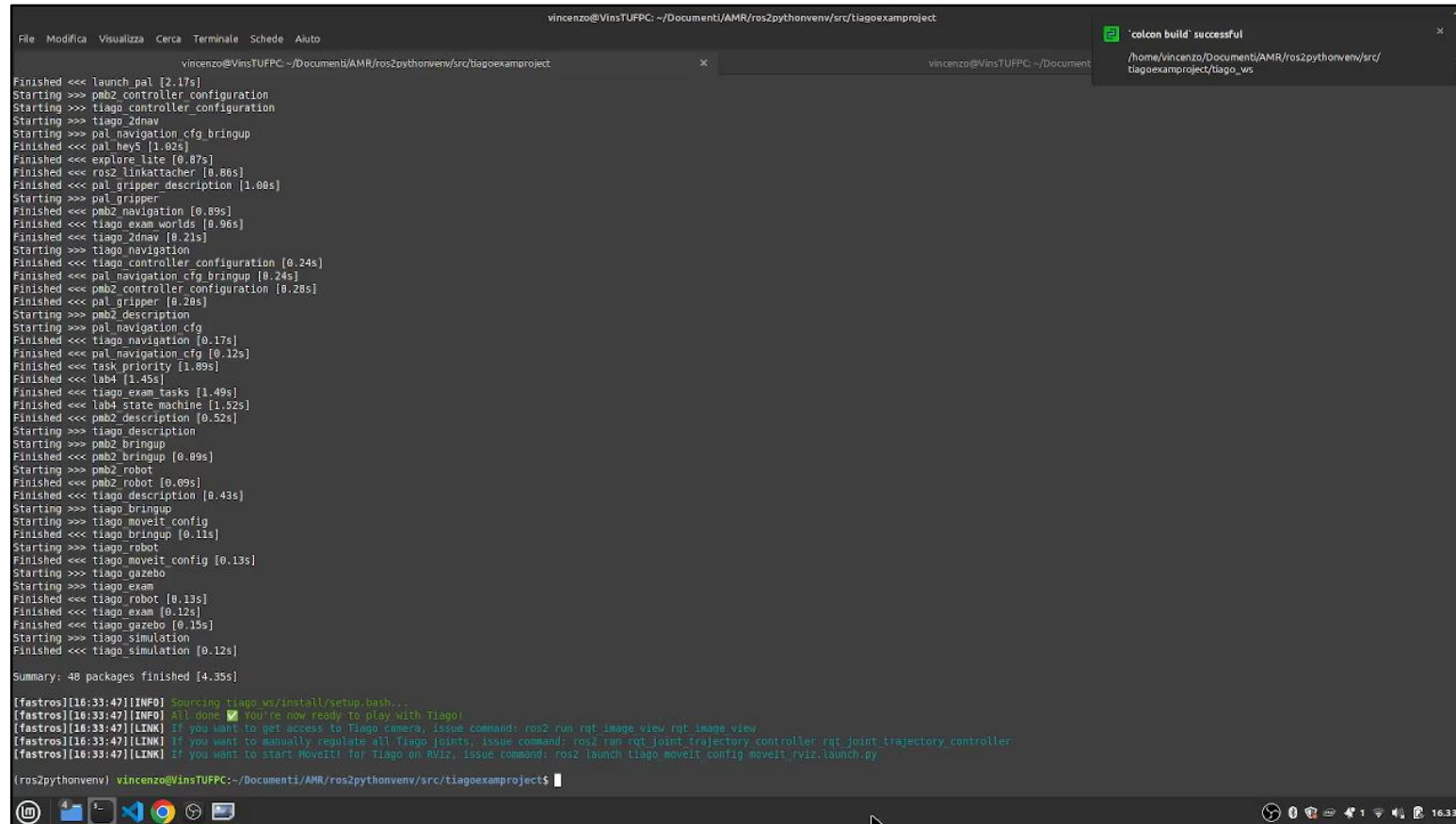
Task2 Behaviour (FSM)

- A simple node implementing the Task3 behaviour as a FSM, exploiting proper topics/services/actions (the usage of these elements as been implemented accordingly to the official ROS2 Humble documentation).

Arm tucking (TiagoArm Action Server)

Autonomous Localiz. (*as before*)

Cubes transportation (pick → place)



```
File Modifica Visualizza Cerca Terminale Schede Aiuto
vincenzo@VinsTUFP: ~/Documenti/AMR/ros2pythonvenv/src/tiagoexamproject
vincenzo@VinsTUFP: ~/Documenti/AMR/ros2pythonvenv/src/tiagoexamproject
Finished <<< launch_pal [2.17s]
Starting >>> pmb2_controller_configuration
Starting >>> tiago_controller_configuration
Starting >>> tiago_2dnav
Starting >>> pal_navigation_cfg_bringup
Finished <<< pal_heys [1.02s]
Finished <<< explore_lite [0.87s]
Finished <<< ros2_LinkAttacher [0.86s]
Finished <<< pal_gripper_description [1.00s]
Starting >>> pal_gripper
Finished <<< pmb2_navigation [0.89s]
Finished <<< tiago_exam_worlds [0.96s]
Finished <<< tiago_2dnav [0.21s]
Starting >>> tiago_navigation
Finished <<< tiago_controller_configuration [0.24s]
Finished <<< pal_navigation_cfg_bringup [0.24s]
Finished <<< pmb2_controller_configuration [0.28s]
Finished <<< pal_gripper [0.28s]
Starting >>> pmb2_description
Starting >>> pal_navigation_cfg
Finished <<< tiago_navigation [0.17s]
Finished <<< pal_navigation_cfg [0.12s]
Finished <<< task_priority [1.09s]
Finished <<< lab4 [1.45s]
Finished <<< tiago_exam_tasks [1.49s]
Finished <<< lab4_state_machine [1.52s]
Finished <<< pmb2_description [0.52s]
Starting >>> tiago_description
Starting >>> pmb2_bringup [0.09s]
Starting >>> pmb2_robot
Finished <<< pmb2_robot [0.09s]
Finished <<< tiago_description [0.43s]
Starting >>> tiago_bringup
Starting >>> tiago_moveitl config
Finished <<< tiago_bringup [0.11s]
Starting >>> tiago_robot
Finished <<< tiago_moveitl config [0.13s]
Starting >>> tiago_gazebo
Starting >>> tiago_exam
Finished <<< tiago_robot [0.13s]
Finished <<< tiago_exam [0.12s]
Finished <<< tiago_gazebo [0.15s]
Starting >>> tiago_simulation
Finished <<< tiago_simulation [0.12s]

Summary: 40 packages finished [4.35s]

[fastros][16:33:47][INFO] Sourcing tiago_ws/install/setup.bash...
[fastros][16:33:47][INFO] All done! You're now ready to play with Tiago!
[fastros][16:33:47][LINK] If you want to get access to Tiago camera, issue command: ros2 run rqt_image_view rqt_image_view
[fastros][16:33:47][LINK] If you want to manually regulate all Tiago joints, issue command: ros2 run rqt_joint_trajectory_controller rqt_joint_trajectory_controller
[fastros][16:33:47][LINK] If you want to start MoveIt! for Tiago on RViz, issue command: ros2 launch tiago_moveitl config moveitl rviz.launch.py

(ros2pythonvenv) vincenzo@VinsTUFP:~/Documenti/AMR/ros2pythonvenv/src/tiagoexamproject$
```

'colcon build' successful
/home/vincenzo/Documenti/AMR/ros2pythonvenv/src/tiagoexamproject/tiago_ws

AMR Tiago Project Report

Vincenzo Lombardi – Group 7
Academic Year 2025/2026

